SPECIAL APPLICATIONS: FACE RECOGNITION AND NEURAL STYLE TRANSFER

Explore how CNNs can be applied to multiple fields, including art generation and face recognition, then implement your own algorithm to generate art and recognize faces!

Learning Objectives

- Differentiate between face recognition and face verification
- Implement one-shot learning to solve a face recognition problem
- Apply the triplet loss function to learn a network's parameters in the context of face recognition
- Explain how to pose face recognition as a binary classification problem
- Map face images into 128-dimensional encodings using a pretrained model
- Perform face verification and face recognition with these encodings
- Implement the Neural Style Transfer algorithm
- Generate novel artistic images using Neural Style Transfer
- Define the style cost function for Neural Style Transfer
- Define the content cost function for Neural Style Transfer

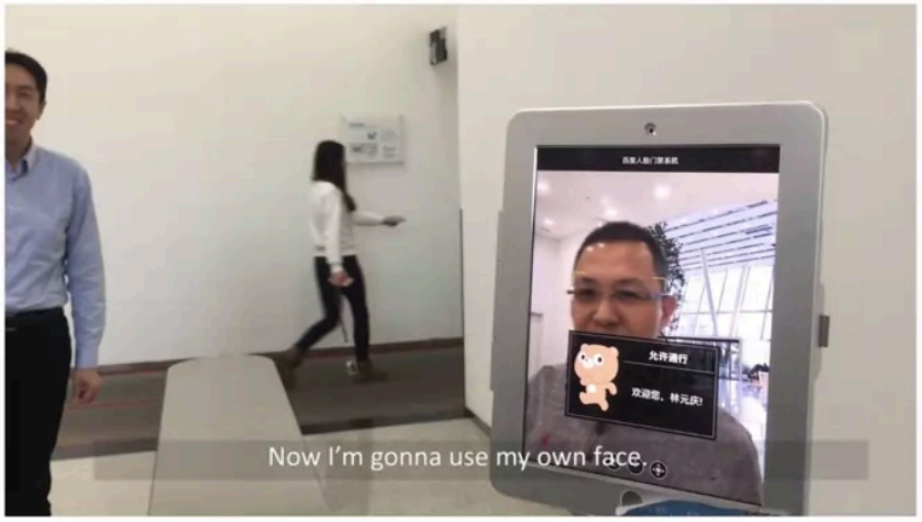


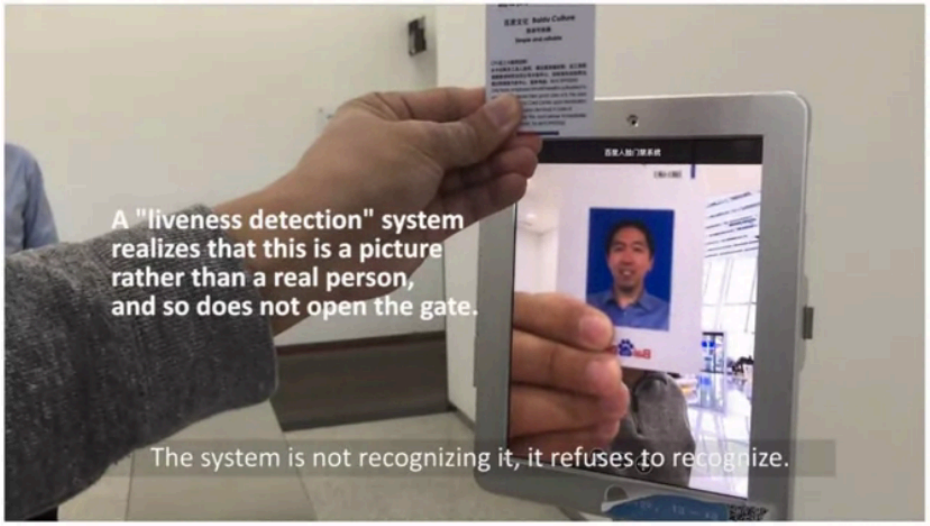
deeplearning.ai

Face recognition

What is face recognition?

Face recognition





Face Recognition

Face Spoofing

(using image of another person)

a.k.a. Liveness detection

[Courtesy of Baidu]

Face verification vs. face recognition

Verification

- Input image, name/ID
- Output whether the input image is that of the claimed person
- 1:1 (one-to-one problem)

Recognition

- Has a database of K persons
- Get an input image
- Output ID if the image is any of the K persons (or "not recognized")

- 1:K problem

Andrew Ng

- Face recognition is a much harder problem than face verification.
  - e.g. Let's say you have a verification system that's 99% accurate.

    Suppose K=100 in a recognition system. If you apply this system to a recognition task with 100 people in the database, you have a hundred times chance of making a mistake, if the chance of making a mistake on each person is just 1%.

    If you want an acceptable recognition error, you might need a verification system with maybe 99.9% or even higher accuracy, before you can run it on a database of 100 people that still have a high chance of getting it correct.



deeplearning.ai

Face recognition

One-shot learning

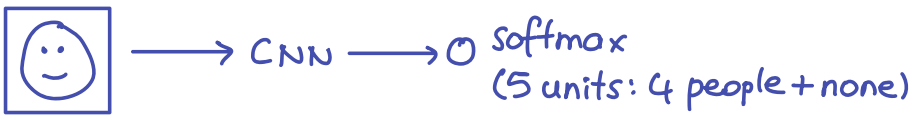
- Goal: Recognize a person given just one single image or just one example of that person's face.
- Historically, deep learning algorithms don't work well if you have only one training example.
- Let's see how to address this problem.

One-shot learning



Learning from one example to recognize the person again

- One approach (which doesn't work):
  Feed the image of a person to a ConvNet:



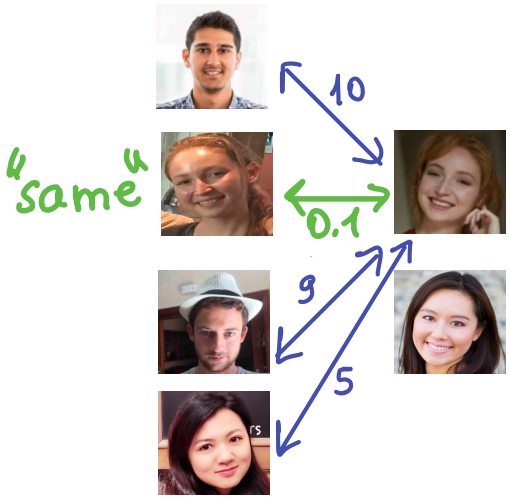
  Problem: with such a small training set, you can't train a CNN!
  - Also, if a new person joins the team, there should now be 6 outputs ⇒ will we re-train the CNN every time? ⇒ not a good approach.

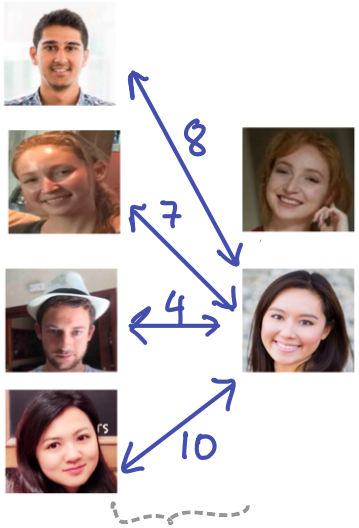
Learning a "similarity" function

d(img1,img2) = degree of difference between images

If d(img1,img2) $\leq \tau$ ⇒ they belong to the "same" person } verification

$> \tau$ ⇒ "different"





large numbers

- If a new person joins, you just add the image to the database.
- How can we learn the function d(.,.) which inputs a pair of images and tells you if they're the same person or different persons?

Andrew Ng



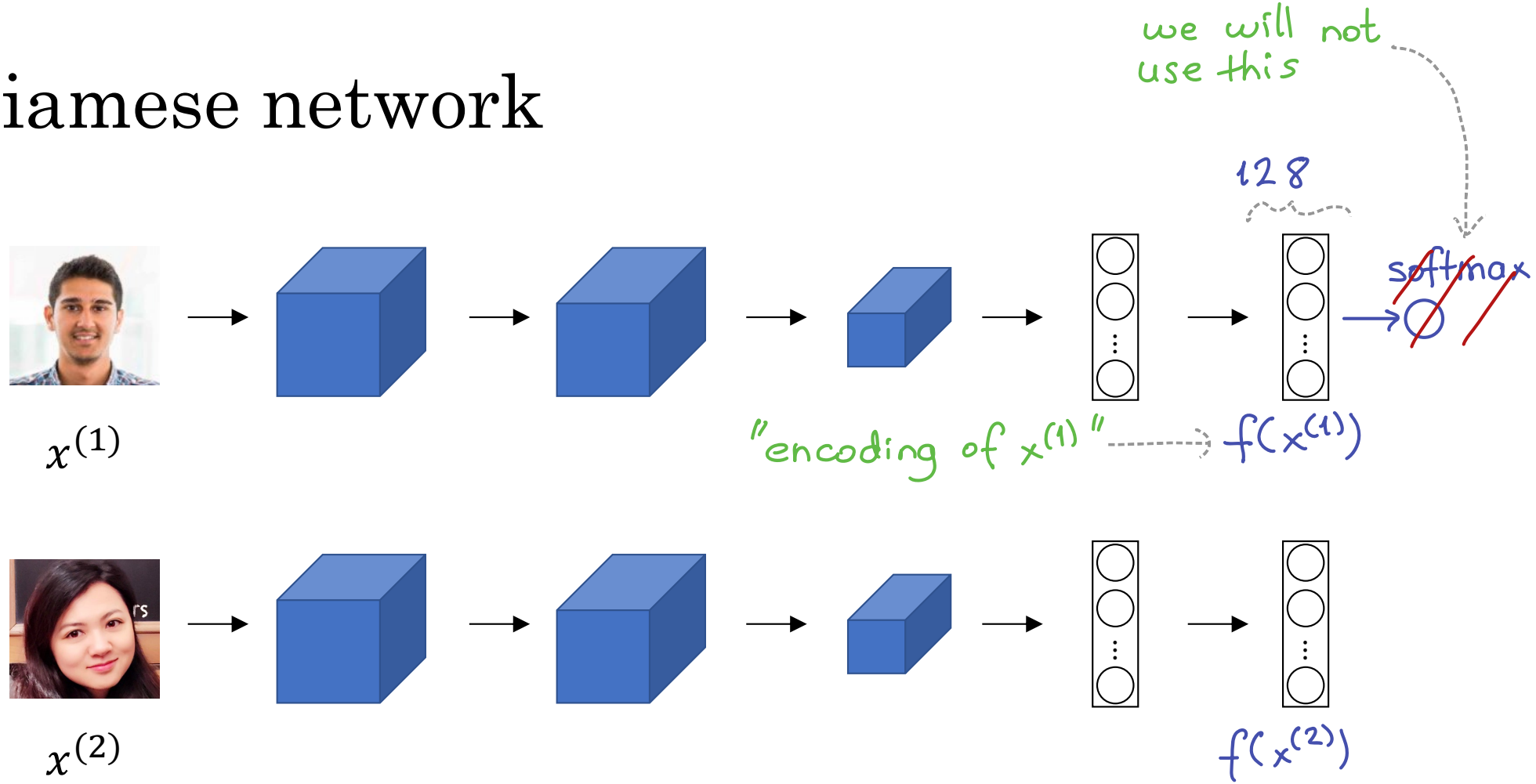
deeplearning.ai

Face recognition

Siamese network

- The difference function d(img1, img2) tells how similar or different img1 and img2 are.
- A good way to do this is to use a Siamese network

Siamese network



[Taigman et. al., 2014. DeepFace closing the gap to human level performance]

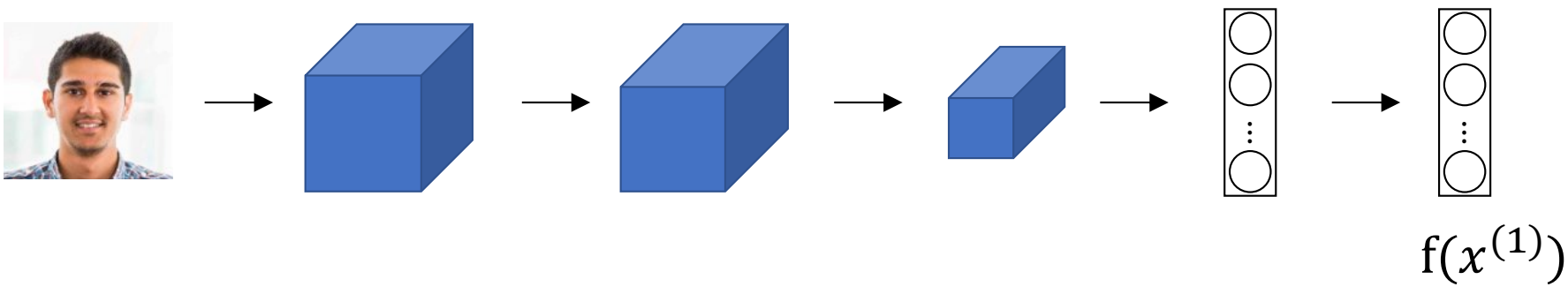
- Top network (previous slide):
  - We will not use the classification layer of the CNN.
  - We're going to focus on the 128 dimensional vector, which is a 128-dim vector representing the image $x^{(1)}$.

- To compare the picture $x^{(2)}$ with $x^{(1)}$, feed $x^{(2)}$ to the same network as $x^{(1)}$, get 128 numbers $f(x^{(2)})$ that represent $x^{(2)}$.
- distance between $x^{(1)}$ and $x^{(2)}$:

$$d(x^{(1)}, x^{(2)}) = \| f(x^{(1)}) - f(x^{(2)}) \|_2^2$$

- Siamese Network: The idea of running two identical CNNs on two different inputs and then comparing them is called the Siamese neural network architecture.
- How do you train a Siamese neural network?

Goal of learning



Parameters of NN define an encoding $f(x^{(i)})$

Learn parameters so that:

If $x^{(i)}, x^{(j)}$ are the same person, $\|f(x^{(i)}) - f(x^{(j)})\|^2$ is small.

If $x^{(i)}, x^{(j)}$ are different persons, $\|f(x^{(i)}) - f(x^{(j)})\|^2$ is large.

} 2 conditions

- Use backpropagation to vary all the parameters in the network to make sure the above 2 conditions are satisfied.
- How do we define an objective function to make a neural network learn to do this?

Andrew Ng



deeplearning.ai

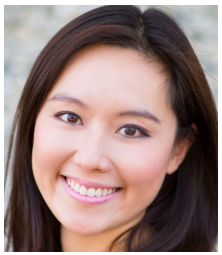
Face recognition

Triplet loss

- One way to learn the parameters of the CNN, so that it gives a good encoding for pictures of faces is to define and apply gradient descent on the triplet loss function.

Learning Objective

- To apply the triplet loss, we need to compare pairs of images.



Anchor

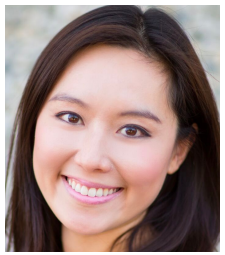
A



Positive

P

- This pair of images belong to the same person.
- We want the distance between these images to be small.



Anchor

A



Negative

N

- This pair of images belong to different people.
- We want the distance between these images to be large.

[Schroff et al.,2015, FaceNet: A unified embedding for face recognition and clustering]

- Triplet loss: we will be looking at 3 images at a time. We will be looking at an anchor image (A), positive image (P), and a negative image (N).

Want: $\underbrace{\|f(A)-f(P)\|^2}_{d(A,P)} \leq \underbrace{\|f(A)-f(N)\|^2}_{d(A,N)}$ (1)

$$\|f(A)-f(P)\|^2 - \|f(A)-f(N)\|^2 \leq 0 \quad (2)$$

Problem: $0 \quad 0 \quad 0 \quad 0 \quad =0$ ✓

if $f(img)=0$ always, you can almost trivially satisfy this equation.

Another problem: If encodings of all images are the same, i.e. $f(A)=f(P)=f(N)$ equation is trivially satisfied

- To prevent the network from doing that, we're going to modify the objective (2) so that, it is not just less than or equal to zero but quite a bit smaller than zero:

$$\|f(A)-f(P)\|^2 - \|f(A)-f(N)\|^2 \leq 0-\alpha$$

or

$$\|f(A)-f(P)\|^2 - \|f(A)-f(N)\|^2 + \alpha \leq 0$$

↳ margin

- Modify equation (1) above as:

$$\underbrace{\|f(A)-f(P)\|^2}_{d(A,P)} + \alpha \leq \underbrace{\|f(A)-f(N)\|^2}_{d(A,N)} \quad (3)$$

↳ 0.2

- If $d(A,P)=0.5$ and $d(A,N)=0.51$ equation (3) will not be satisfied.

$$\underbrace{d(A,P)}_{0.5} + 0.2 \leq \underbrace{d(A,N)}_{0.7 \rightarrow \text{must be at least } 0.7}$$

↳ may decrease this or ↑

- The margin pushes the anchor-positive pair and the anchor-negative pair further away from each other.

Loss function

Given 3 images A, P, N: (A, P: same; P, N: different)

$$\mathcal{L}(A,P,N) = \max\left(\|f(A)-f(P)\|^2 - \|f(A)-f(N)\|^2 + \alpha, 0\right)$$

: loss on a single triplet

we want this to be ≤ 0

- if it is $\leq 0 \Rightarrow$ loss $= 0$
- if it is $> 0 \Rightarrow$ loss will be positive

Overall cost function:

$$J = \sum_{i=1} \mathcal{L}(A^{(i)}, P^{(i)}, N^{(i)})$$

: loss on the $(i)^{th}$ triplet

Training set: 10k pictures of 1k persons

Generate triplets (A, P, N), then train the network using gradient descent.

[Schroff et al.,2015, FaceNet: A unified embedding for face recognition and clustering]

- To define triplets, we need pairs like A, P, pairs of pictures of the same person. Hence, we need a dataset where we have multiple pictures of the same person. (If you had just 1 picture of each person, then we can't train this system).
- After having trained a system, you can then apply it to the one-shot learning problem, where for your face recognition system, you have only a single picture of someone you are trying to recognize.

- Next: how do we choose the triplets from the training dataset?

Choosing the triplets A,P,N

During training, if A,P,N are chosen randomly, $d(A,P) + \alpha \leq d(A,N)$ is easily satisfied.

$\|f(A) - f(P)\|^2 + \alpha \leq \|f(A) - f(N)\|^2$

Choose triplets that're "hard" to train on.

$d(A,P) + \alpha \leq d(A,N)$

A "hard" triplet: $d(A,P) \approx d(A,N)$ ⇒ algorithm will either decrease $d(A,P)$ or increase $d(A,N)$ to satisfy the margin constraint.

[Schroff et al.,2015, FaceNet: A unified embedding for face recognition and clustering]

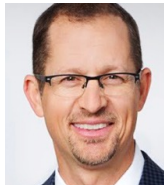
- Choosing "hard" triplets increases computational efficiency of the learning algorithm.
- If you choose the triplets randomly, then too many triplets would be easy and gradient descent won't do anything because the NN would get them right pretty much all the time.

- Naming algorithms in deep learning:

  Face Net

  Deep Face

Training set using triplet loss

Anchor	Positive	Negative
		
		
⋮	⋮	⋮
		

- After defining the triplets use gradient descent to minimize the cost function J so that $d(x^{(i)}, x^{(j)})$ will be small if $x^{(i)}$ and $x^{(j)}$ belong to the same person, and it will be large if they belong to different people.

Andrew Ng

- Face recognition systems are trained on very large datasets (10 M, 100 M images).
- These datasets are not easy to acquire.
- Some companies and research groups have trained these large networks and posted parameters online.
- Rather than training a network from scratch, you can download a pre-trained model.



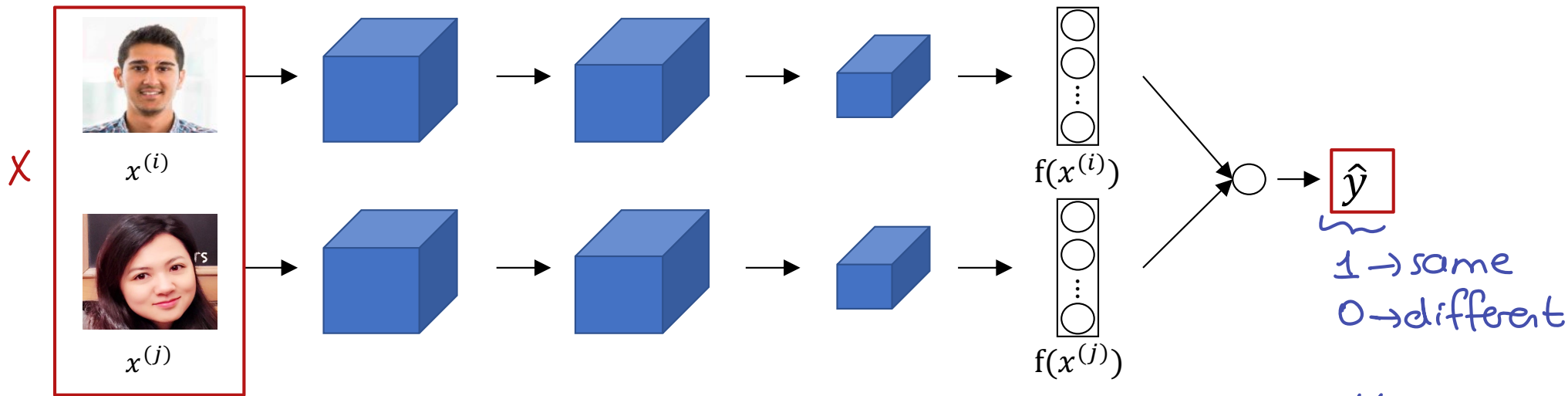
deeplearning.ai

Face recognition

Face verification and binary classification

- Face recognition can also be posed as a straight binary classification.

Learning the similarity function



- Take the Siamese network and have them both compute the embeddings $f(x^{(i)})$ and $f(x^{(j)})$ (maybe 128 or higher dimensional embeddings).
- Input the embeddings to a logistic regression unit and make a prediction (1: same, 0: different)

[Taigman et. al., 2014. DeepFace closing the gap to human level performance]

- This binary classification method is an alternative to triplet loss for training a system like this.

- What does this final logistic regression unit do?
  - The output $\hat{y}$ will be a sigmoid function applied to some set of features.
  - But rather than just feeding in the encodings, you can take the differences between the encodings

$$\hat{y} = \sigma\left(\sum_{k=1}^{128} w_k \left| f(x^{(i)})_k - f(x^{(j)})_k \right| + b \right)$$

logistic regression parameters

- Another variation (instead of the absolute difference:

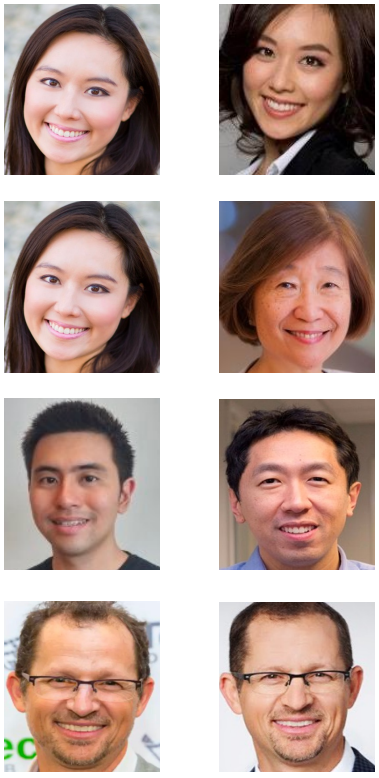
$$\frac{\left(f(x^{(i)})_k - f(x^{(j)})_k\right)^2}{f(x^{(i)})_k + f(x^{(j)})_k}$$: χ^2, chi-square similarity

- In the above network, the input is a pair of images $x: (x^{(i)}, x^{(j)})$ and $\hat{y}$ (0 or 1).
- You're training a Siamese network, which means that the parameters of the upper network are "tied" to the parameters of the lower network.
- This system works pretty well.
- A computational trick:
  - Let $x^{(i)}$: new image, $x^{(j)}$: an image in database
  - Instead of computing $x^{(j)}$ every single time, you can pre-compute $x^{(j)}$ and a new person comes you can compute and compare $x^{(i)}$ and $x^{(j)}$ and make a prediction $\hat{y}$.
  - So, you don't need to store the raw images, and if you

have a very large database of employees, you don't need to compute the encodings every single time for every employee in the database.

- This idea of pre-computing also works for the triplet-loss function based method.

Face verification supervised learning

x	y	
	1	"same"
	0	"different"
	0	
	1	

- Use these pairs to train a NN using backpropagation.

[Taigman et. al., 2014. DeepFace closing the gap to human level performance]



deeplearning.ai

Neural Style Transfer

What is neural style transfer?

Neural style transfer

"Starry Night"





Content (C) (S) Style

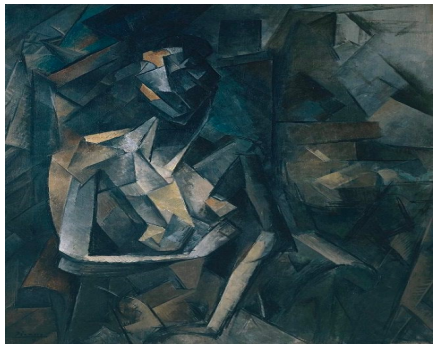
Van Gogh



Generated image (G)

Pablo Picasso





Content (C) Style (S)



Generated image (G)

[Images generated by Justin Johnson]

Andrew Ng



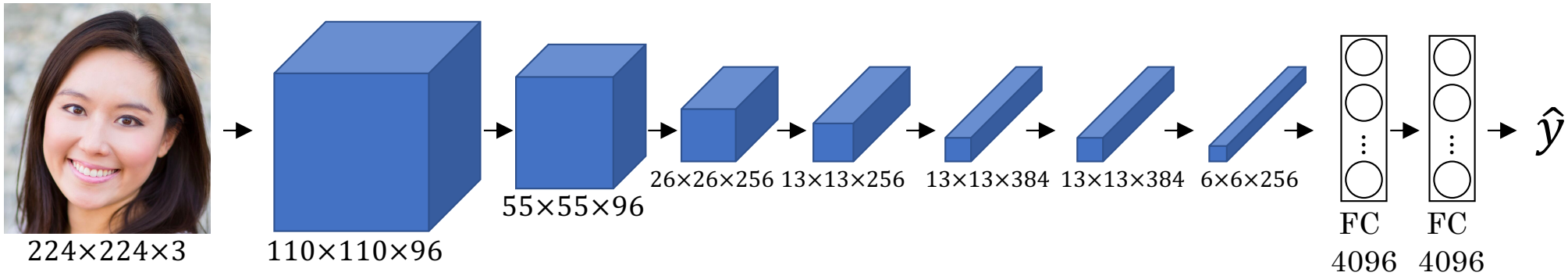
deeplearning.ai

Neural Style Transfer

What are deep ConvNets learning?

Goal: Visualize what the hidden units in different layers are computing.

Visualizing what a deep network is learning



Pick a unit in layer 1. Find the nine image patches that maximize the unit's activation.

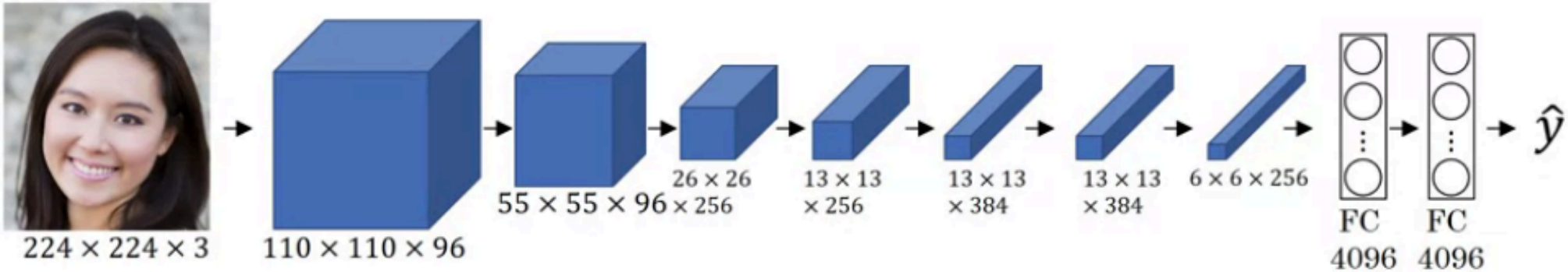
Repeat for other units.

[Zeiler and Fergus., 2013, Visualizing and understanding convolutional networks]

- Start with a hidden unit in layer 1. Suppose you can scan through your training set and find out what are the imager or image patches that maximize that unit's activation.
- i.e. pass your training set through your neural network, and figure out what is the image that maximizes that particular unit's activation.

- A hidden unit in layer 1, will only see a relatively small portion of the image. If you plot what activated the unit, it makes sense to plot just small image patches, because that's all of the image that that particular unit sees.

Visualizing what a deep network is learning



Pick a unit in layer 1. Find the nine image patches that maximize the unit's activation.

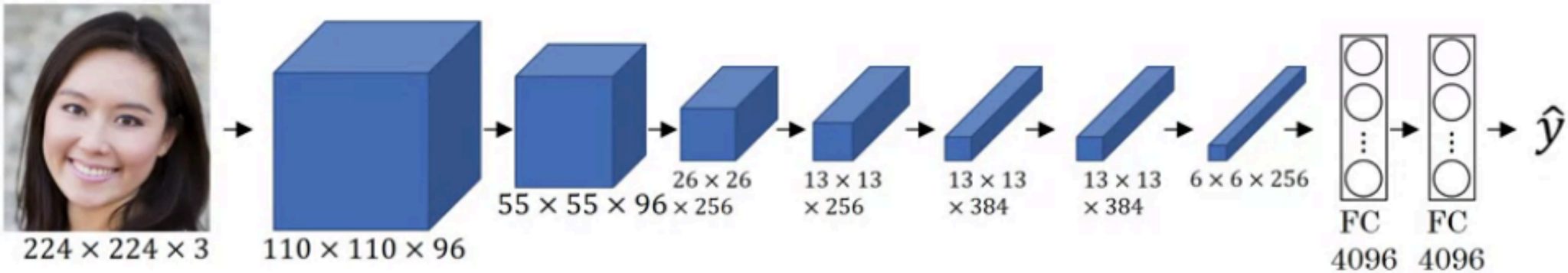


The hidden unit is looking for edges that looks like:



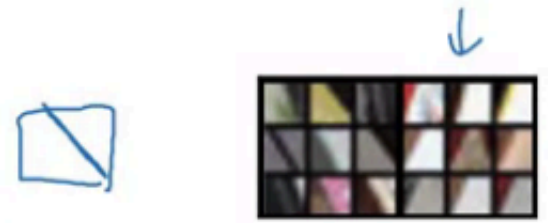
Zeiler and Fergus., 2013, Visualizing and understanding convolutional networks]

Visualizing what a deep network is learning



Pick a unit in layer 1. Find the nine image patches that maximize the unit's activation.

Repeat for other units.

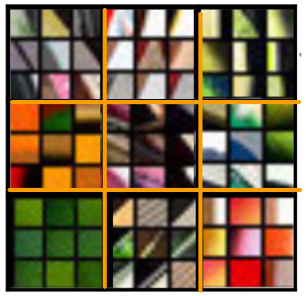


looking for edges like this.

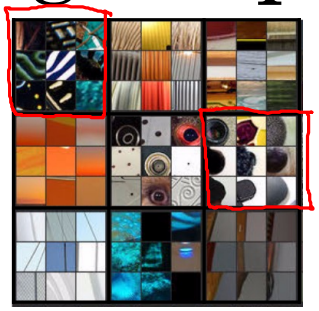
9 image patches that activate the second unit.

Zeiler and Fergus., 2013, Visualizing and understanding convolutional networks]

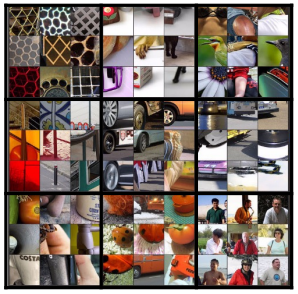
Visualizing deep layers



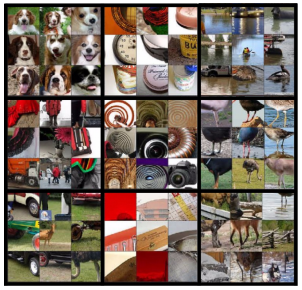
Layer 1



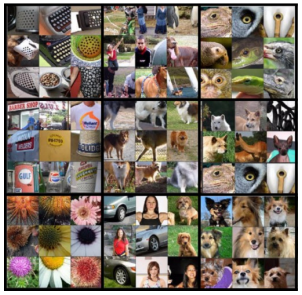
Layer 2



Layer 3



Layer 4



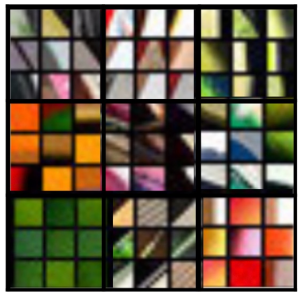
Layer 5

- 9 hidden units and the 9 image patches that activate them.
- Units in hidden layer 1 look for simple features (edges or a shade of color)
- In deeper layers, a hidden unit will see a larger region of the image.
- More complex patterns are detected in deeper layers.
- The image patches will be plot the same size as before

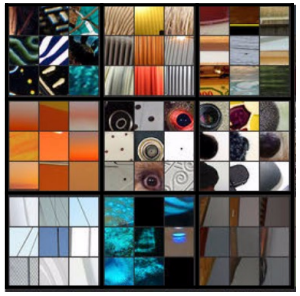
* • 9 image patches that activate hidden unit 1 in layer 2.

Andrew Ng

Visualizing deep layers: Layer 1



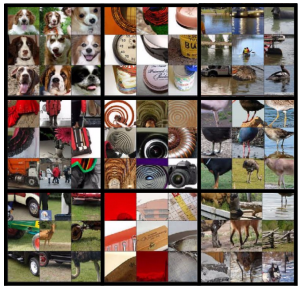
Layer 1



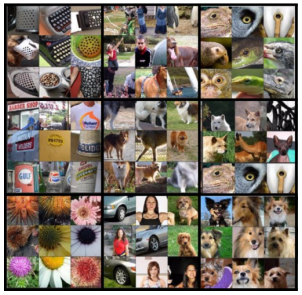
Layer 2



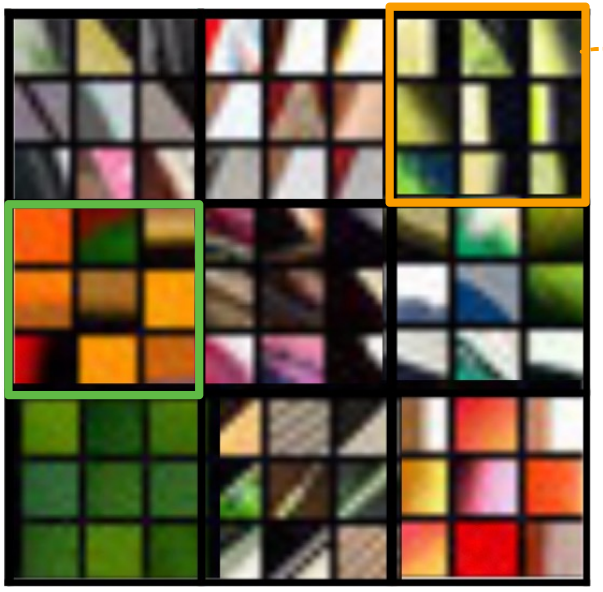
Layer 3



Layer 4



Layer 5

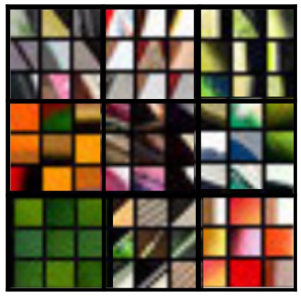


This hidden unit detects vertical edges where the left side is light green

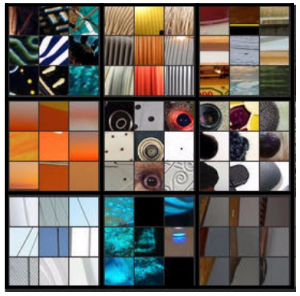
orange patches

Andrew Ng

Visualizing deep layers: Layer 2



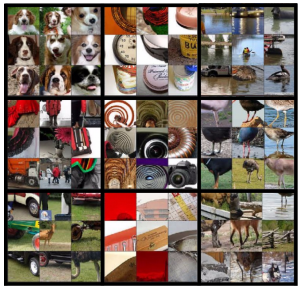
Layer 1



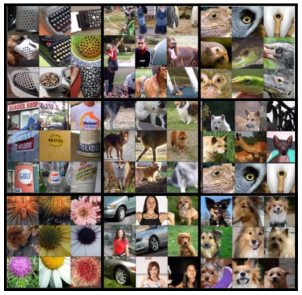
Layer 2



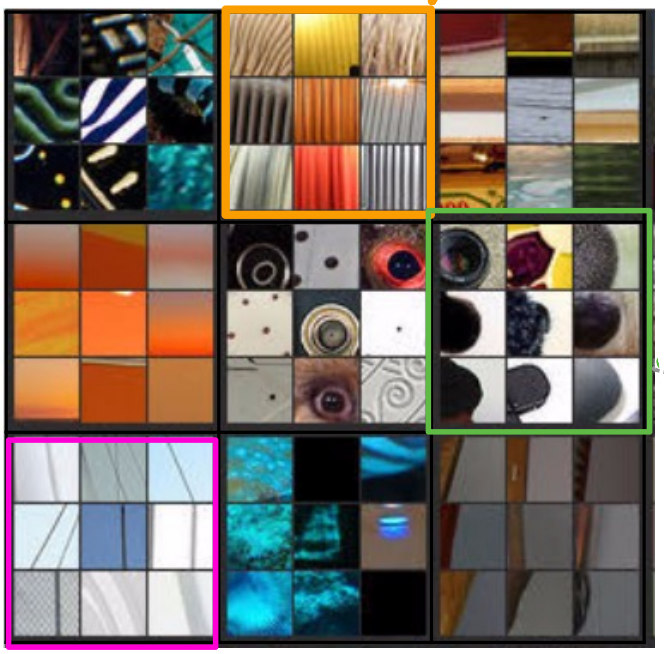
Layer 3



Layer 4



Layer 5



- More complex shapes and patterns are detected.

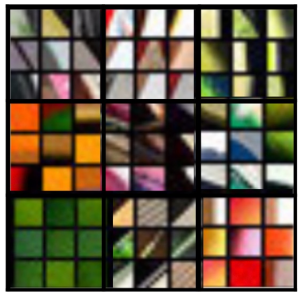
A • This hidden unit is detecting vertical stripes.

- This hidden unit is activated when there's a round shape on the left part.

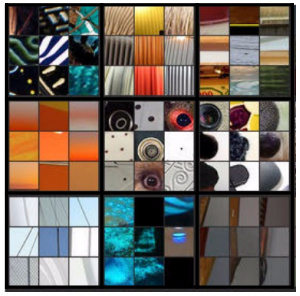
C • This one is looking for thin vertical lines.

Andrew Ng

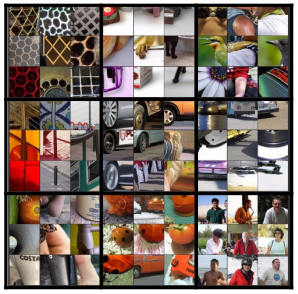
Visualizing deep layers: Layer 3



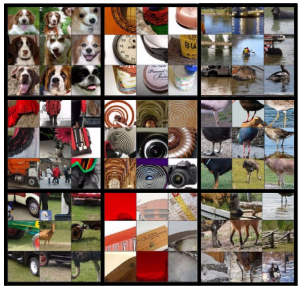
Layer 1



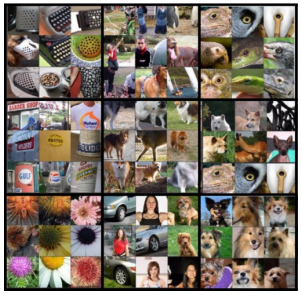
Layer 2



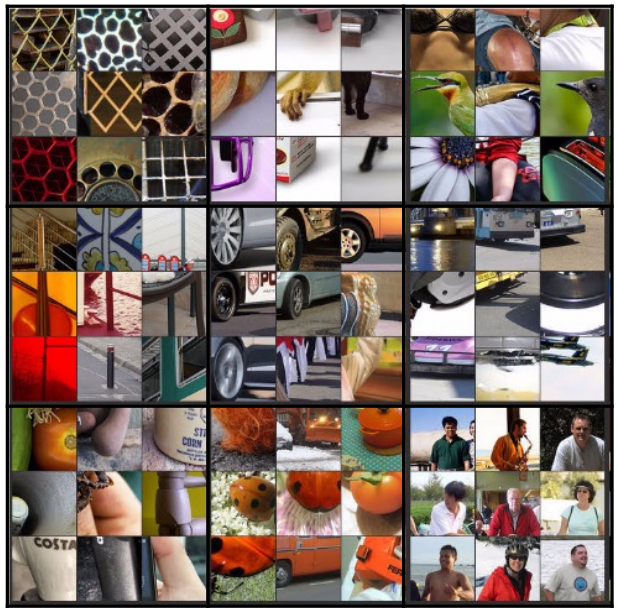
Layer 3



Layer 4



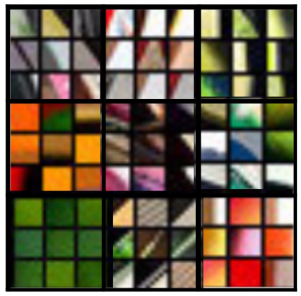
Layer 5



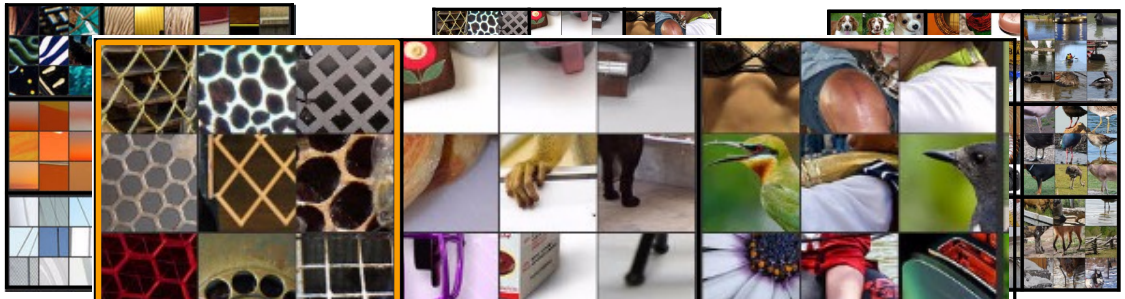
Patches that maximally activate hidden units in layer 3.

Andrew Ng

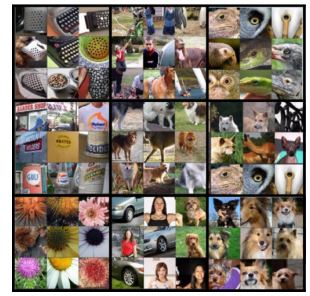
Visualizing deep layers: Layer 3



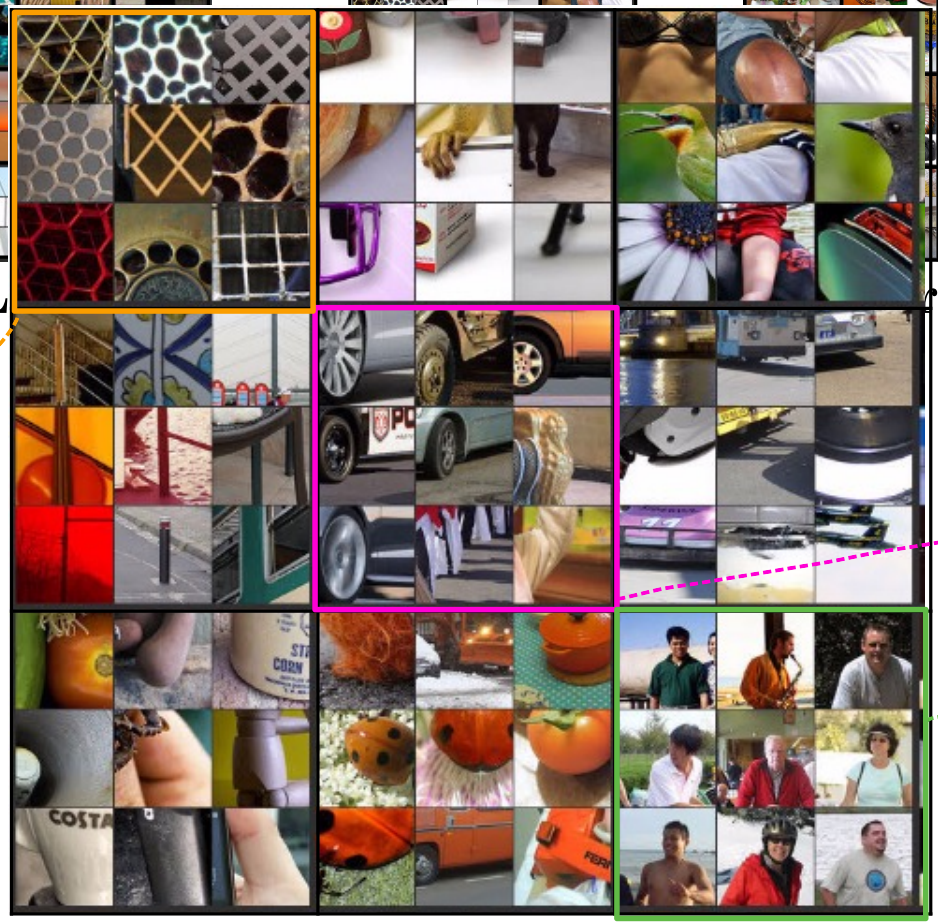
Layer 1



L … 4



Layer 5



Detecting Textures (honey comb shaper etc.)

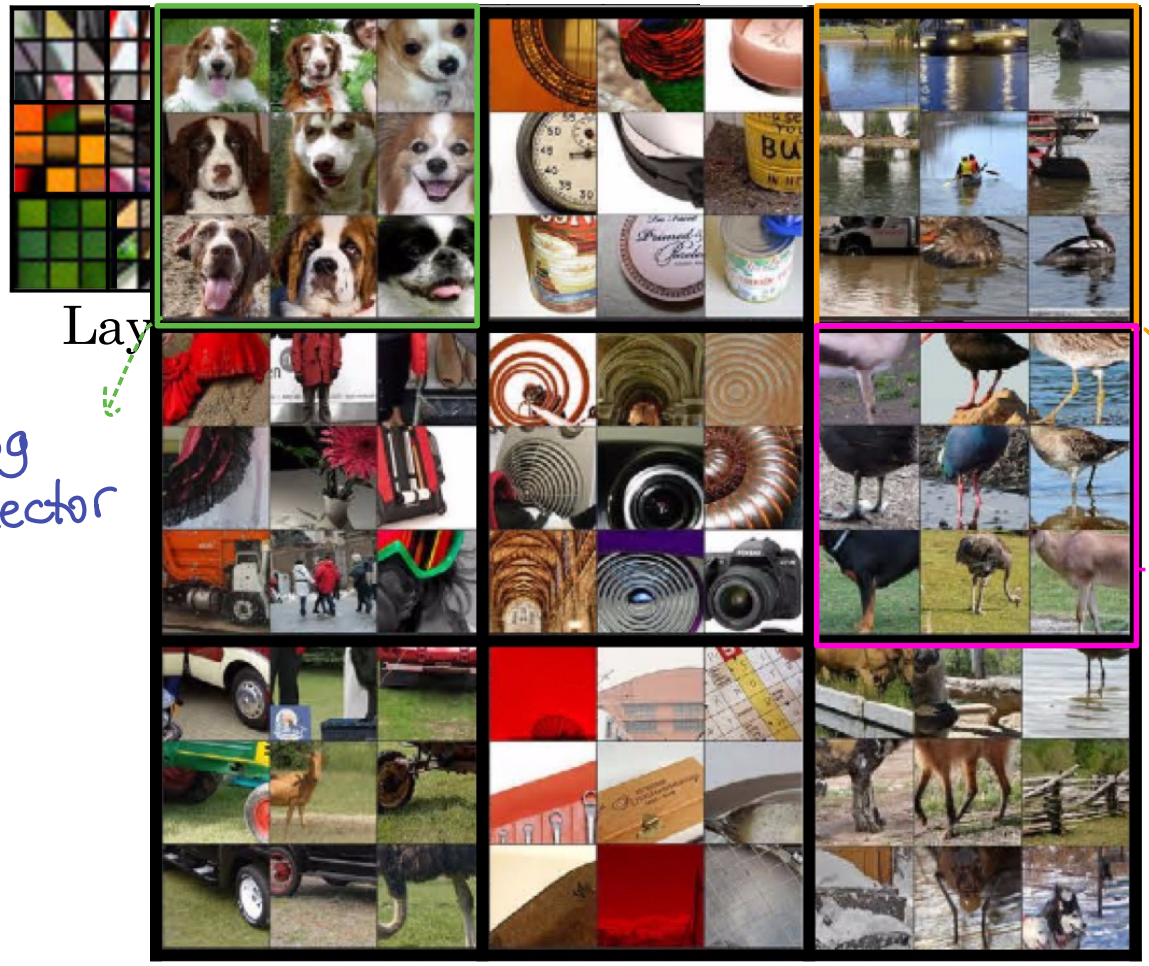
Detecting round shaper in the left part (car tires)

Detecting people

More complex patterns are detected.

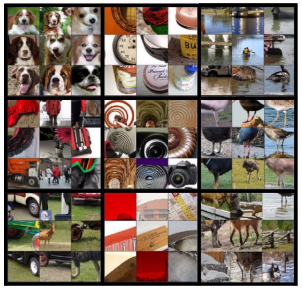
Andrew Ng

Visualizing deep layers: Layer 4

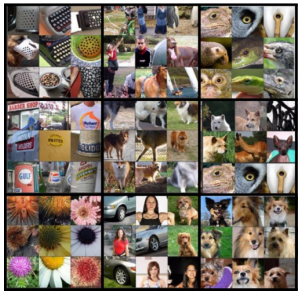


Lay

Layer 4



Layer 5



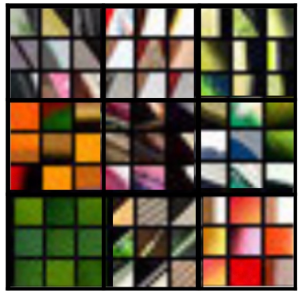
Dog detector

This hidden unit is detecting water

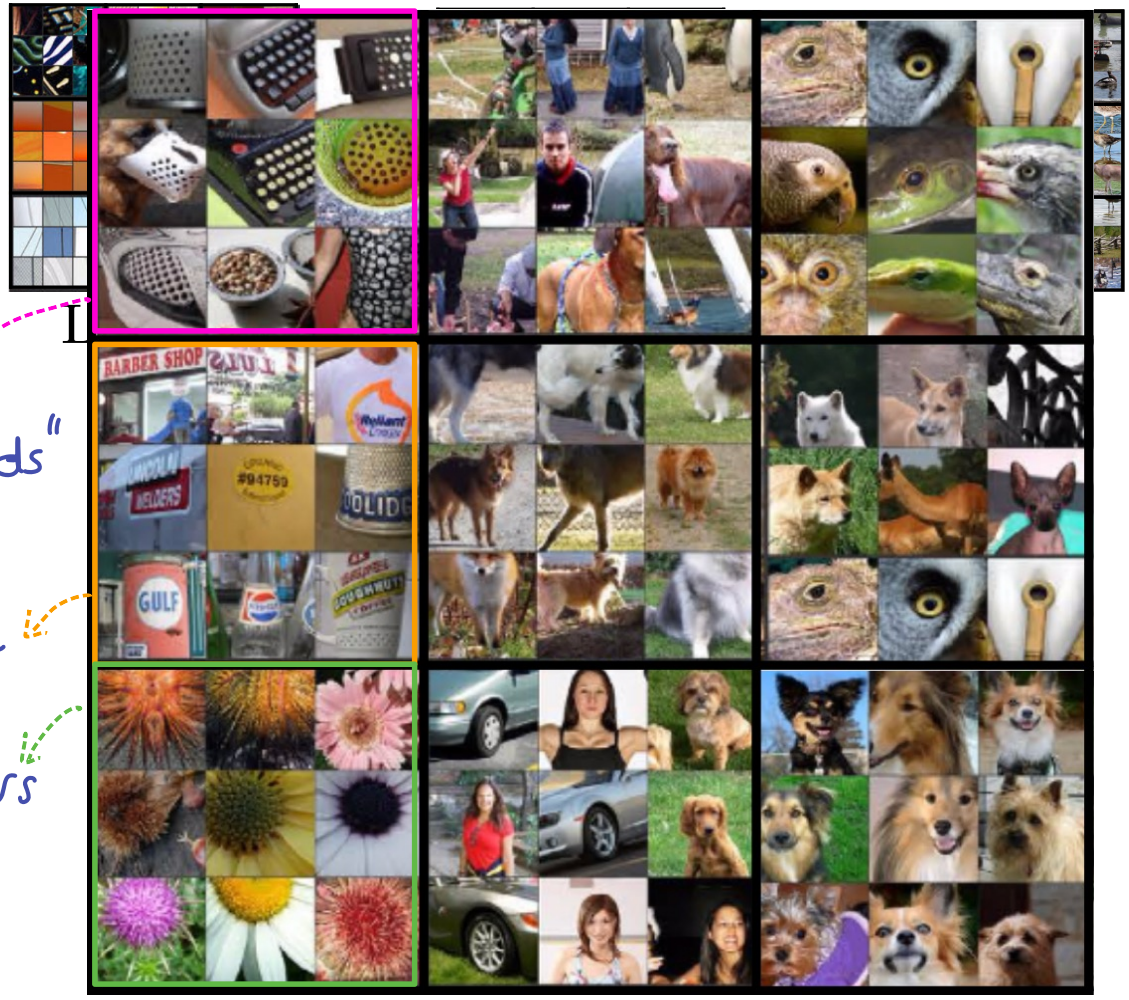
Detecting legs of a bird.

Andrew Ng

Visualizing deep layers: Layer 5



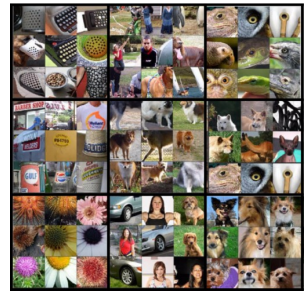
Layer 1



"Detecting keyboards" like texture.

Detecting text

Detecting flowers



Layer 5

Andrew Ng



deeplearning.ai

Neural Style Transfer

Cost function

Neural style transfer cost function



Content C



Style S



Generated image G

- Goal: Define a cost function $J(G)$ that measures how good the generated new image G is.
- Then use gradient descent to minimize $J(G)$.

$$J(G) = \alpha J_{content}(C,G) + \beta J_{style}(S,G)$$

$J_{content}(C,G)$: measures the similarity of C and G images

$J_{style}(S,G)$: measures the similarity of S and G images

α, β: weights

[Gatys et al., 2015. A neural algorithm of artistic style. Images on slide generated by Justin Johnson]

Find the generated image G

1. Initiate G randomly

   G: 100×100×3

2. Use gradient descent to minimize $J(G)$

   $G := G - \frac{\partial}{\partial G} J(G)$ (update pixels of G)

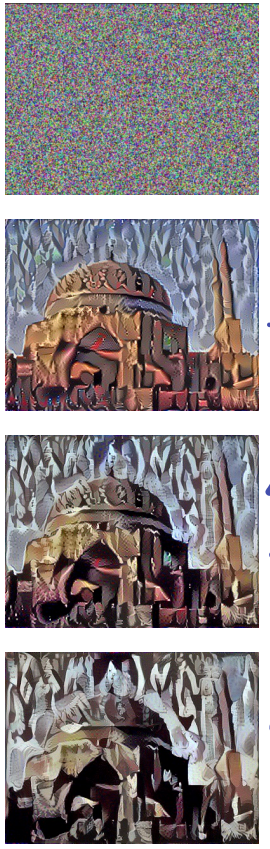
C



S



G: initialized



iterate

[Gatys et al., 2015. A neural algorithm of artistic style]



deeplearning.ai

Neural Style Transfer

Content cost function

Content cost function

$$J(G) = \alpha\, J_{content}(C, G) + \beta\, J_{style}\,(S, G)$$

- Say you use hidden layer l to compute content cost.
- Use pre-trained ConvNet. (E.g., VGG network)
- Let $a^{[l](C)}$ and $a^{[l](G)}$ be the activation of layer l on the images
- If $a^{[l](C)}$ and $a^{[l](G)}$ are similar, both images have similar content
- $J_{content}(C,G) = \frac{1}{2} \| a^{[l]C} - a^{[l](G)} \|^2$

reshaped into vectors

[Gatys et al., 2015. A neural algorithm of artistic style]

- How to define the content cost function:
  - Let's say you use hidden layer l to compute the content cost.
  - If l is a small number, (e.g if you use hidden layer 1), it will really force your generated image pixel values to be very similar to content image.
  - If you use a very deep layer, then it is asking, "well, if there's a dog in your content image, then make sure there is a dog somewhere in your generated image."
  - In practice, layer l is chosen somewhere in between. It is neither too shallow or too deep in the neural network.
  - Given a content image and given a generated image, how similar are they in contents?

  - $J_{content}(C,G) = \frac{1}{2} \| a^{[l](C)} - a^{[l](G)} \|^2$

    activations of layer [l] reshaped into vectors

  - When you perform gradient descent on $J(G)$ to find the value of G, so that the overall cost is low, this will encourage the algorithm to find an image G, so that hidden layer activations are similar to what you got in the content image.

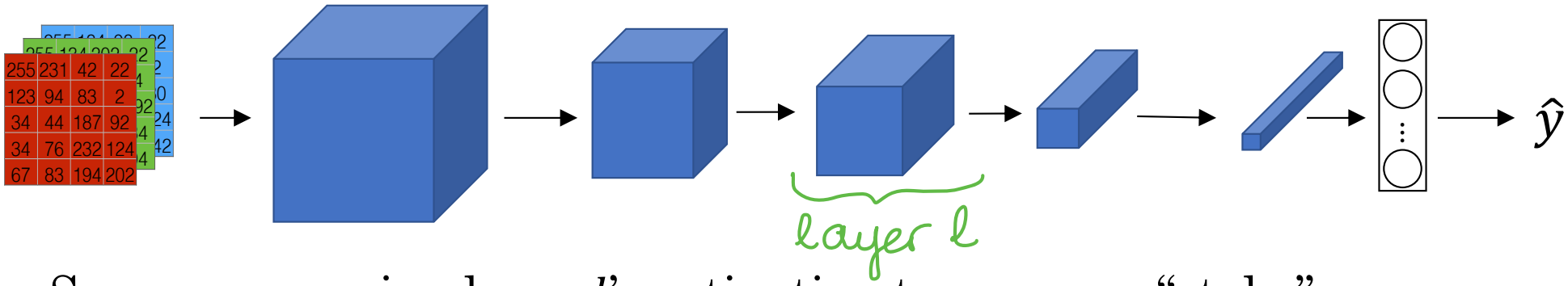


deeplearning.ai

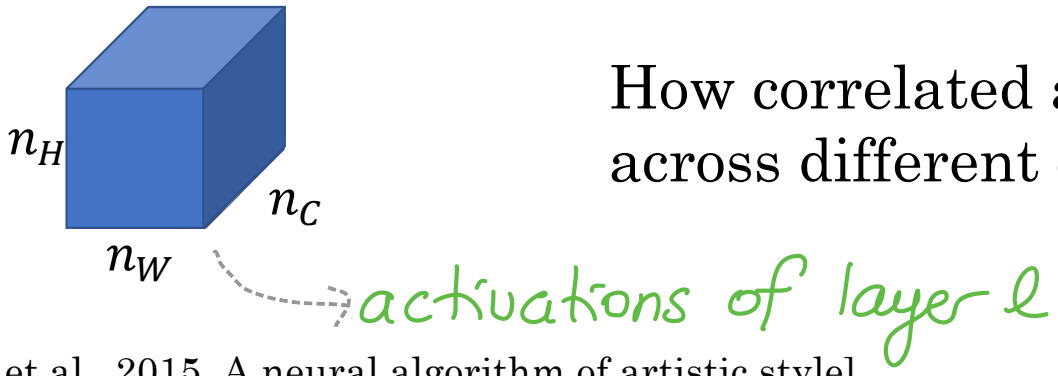
Neural Style Transfer

Style cost function

Meaning of the "style" of an image



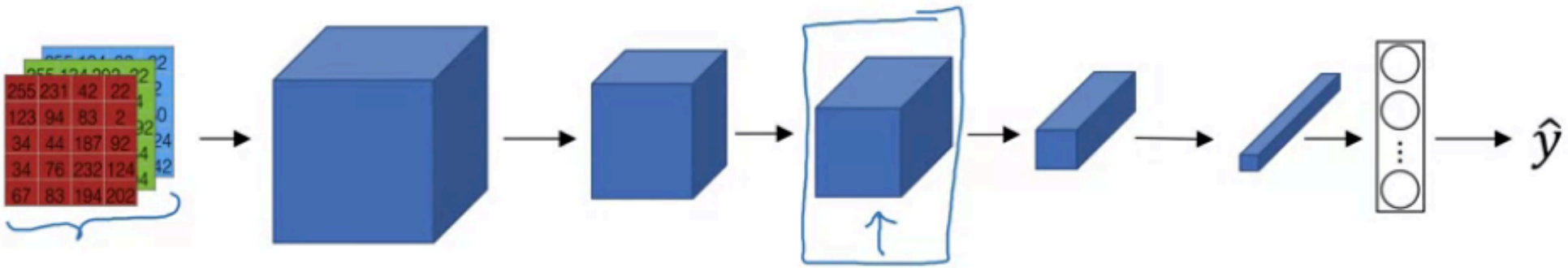
Say you are using layer l's activation to measure "style."
Define style as correlation between activations across channels.



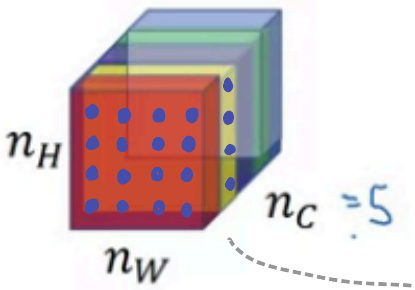
How correlated are the activations across different channels?

[Gatys et al., 2015. A neural algorithm of artistic style]

Meaning of the "style" of an image



Say you are using layer l's activation to measure "style."
Define style as correlation between activations across channels.



How correlated are the activations across different channels?

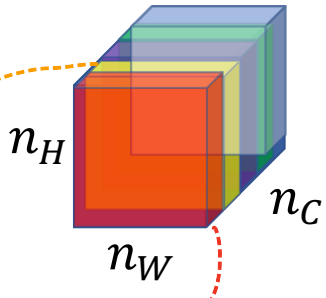
eg. Look at the red and yellow channels. Take the pairs of numbers in the red and yellow channels and measure how correlated they are.

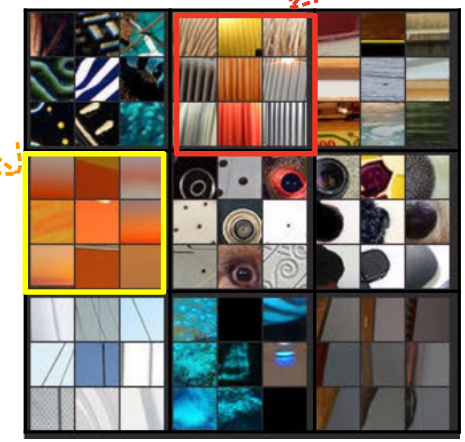
Why does this capture style?

[Gatys et al., 2015. A neural algorithm of artistic style]

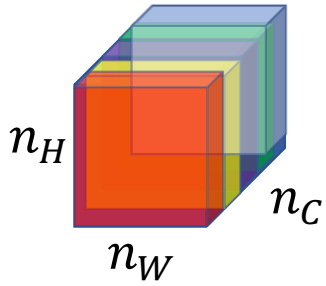
Intuition about style of an image

Style image





Generated Image



- Let's say red channel corresponds to the red neuron trying to figure out if there are vertical stripes in a particular position.
- Let the yellow channel correspond to the yellow neuron which is looking for orange colored patches.
- What does it mean for these two channels to be highly correlated?

[Gatys et al., 2015. A neural algorithm of artistic style]

- what does it mean for these two channels to be highly correlated?
- If the red and yellow channels are highly correlated, it means that whenever part of the image has the subtle vertical texture, that part of the image will also have the orange tint in color.
- what does it mean for them to be uncorrelated?
- whenever there's this vertical texture, it probably won't have the orange tint.
- Correlation tells you which of these high level texture components tend to occur or not occur together.
- Degree of correlation between channels gives you a way of measuring how often these different high level features, such as vertical texture and orange tint, occur together, and don't occur together in different parts of the style image.
- We can use the degree of correlation between channels as a measure of the style.
- Then, you can measure the degree of correlations in your generated image similarly.
- Comparing the correlations in the style and generated images will give you a measure of how similar is the style of the generated image to the style of the input style image.

Style matrix ($G^{[l]}$): measure of correlations

Let $a^{[l]}_{i,j,k}$ = activation at (i, j, k). $G^{[l]}$ is $n_c^{[l]} \times n_c^{[l]}$

i, j, k are indices for H (height), W (width) and C (channels).

n_c: # channels

$G^{[l]}_{kk'}$: measure of correlation between activations in channel k compared to the activations in channel k', at layer l.

$k, k' = 1, \ldots, n_c^{[l]}$

[Gatys et al., 2015. A neural algorithm of artistic style]

Style matrix is computed as:

$$G^{[l]}_{kk'} = \sum_{i=1}^{n_H} \sum_{j=1}^{n_W} a^{[l]}_{ijk} \, a^{[l]}_{ijk'} \quad , \quad k, k' = 1, \ldots, n_c^{[l]} \qquad (1)$$

- Compute the above value for all k, k' values to obtain the style matrix $G^{[l]}$, which is $n_c^{[l]} \times n_c^{[l]}$.
- Note: Equation (1) is unnormalized cross-covariance, since we're not subtracting the mean and multiplying the elements directly.
- Note: (1) is also called as the "Gram matrix".
- Compute (1) for both the style and generated images.

$$G^{[l](S)}_{kk'} = \sum_{i=1}^{n_H} \sum_{j=1}^{n_W} a^{[l](S)}_{ijk} \, a^{[l](S)}_{ijk'}$$

$$G^{[l](G)}_{kk'} = \sum_{i=1}^{n_H} \sum_{j=1}^{n_W} a^{[l](G)}_{ijk} \, a^{[l](G)}_{ijk'}$$

$$J^{[l]}_{style}(S, G) = \left\| G^{[l](S)} - G^{[l](G)} \right\|_F^2 \quad : \text{Frobenius norm of differences}$$

$$= \frac{1}{(2 n_H^{[l]} n_W^{[l]} n_c^{[l]})^2} \sum_k \sum_{k'} \left(G^{[l](S)}_{kk'} - G^{[l](G)}_{kk'} \right)^2$$

normalization constant

(not that important since J_{style} is multiplied with β afterwards)

Style cost function

$$J_{style}^{[l]}(S,G) = \frac{1}{\left(2n_H^{[l]}n_W^{[l]}n_C^{[l]}\right)^2}\sum_k\sum_{k'}\left(G_{kk'}^{[l](S)} - G_{kk'}^{[l](G)}\right)^2$$ (for layer $[l]$)

$$J_{style}(S,G) = \sum_l \lambda^{[l]} J_{style}^{[l]}(S,G)$$ (overall style cost function)

- You get more visually pleasing results if you use the style cost function from multiple different layers.
- $\lambda^{[l]}$: hyperparameter. Allows you to use different layers in a neural network. The early layers measure relatively simpler low level features like edges, as well as later layers, which measure high-level features.

[Gatys et al., 2015. A neural algorithm of artistic style]

- Therefore, both low-level and high-level correlations are taken into account when computing style.

Overall cost function is defined as:

$$J(G) = \alpha J_{content}(C, G) + \beta J_{style}(S, G)$$

- Use gradient descent based optimization to minimize $J(G)$ w.r.t. G (generated image).

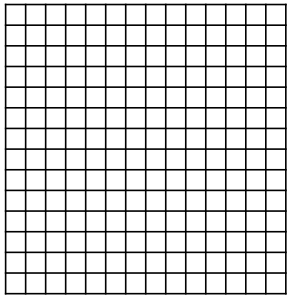


deeplearning.ai

Convolutional Networks in 1D or 3D

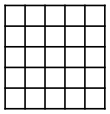
1D and 3D generalizations of models

Convolutions in 2D and 1D



2D input image
14×14

*

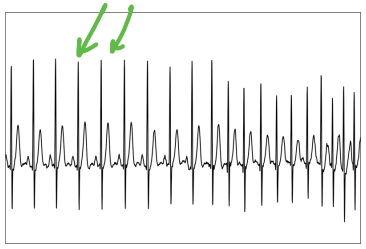


2D filter
5×5

:

input (14×14×3) * 16 filters (5×5×3) → output (10×10×16)

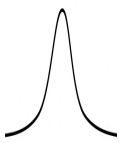
EKG



Heart beats

1	20	15	3	18	12	4	17

*



1	3	10	3	1

:

16 filters
(14×1) * (5×1) → (10×16)

32 filters
(10×16) * (5×1) → (6×32)

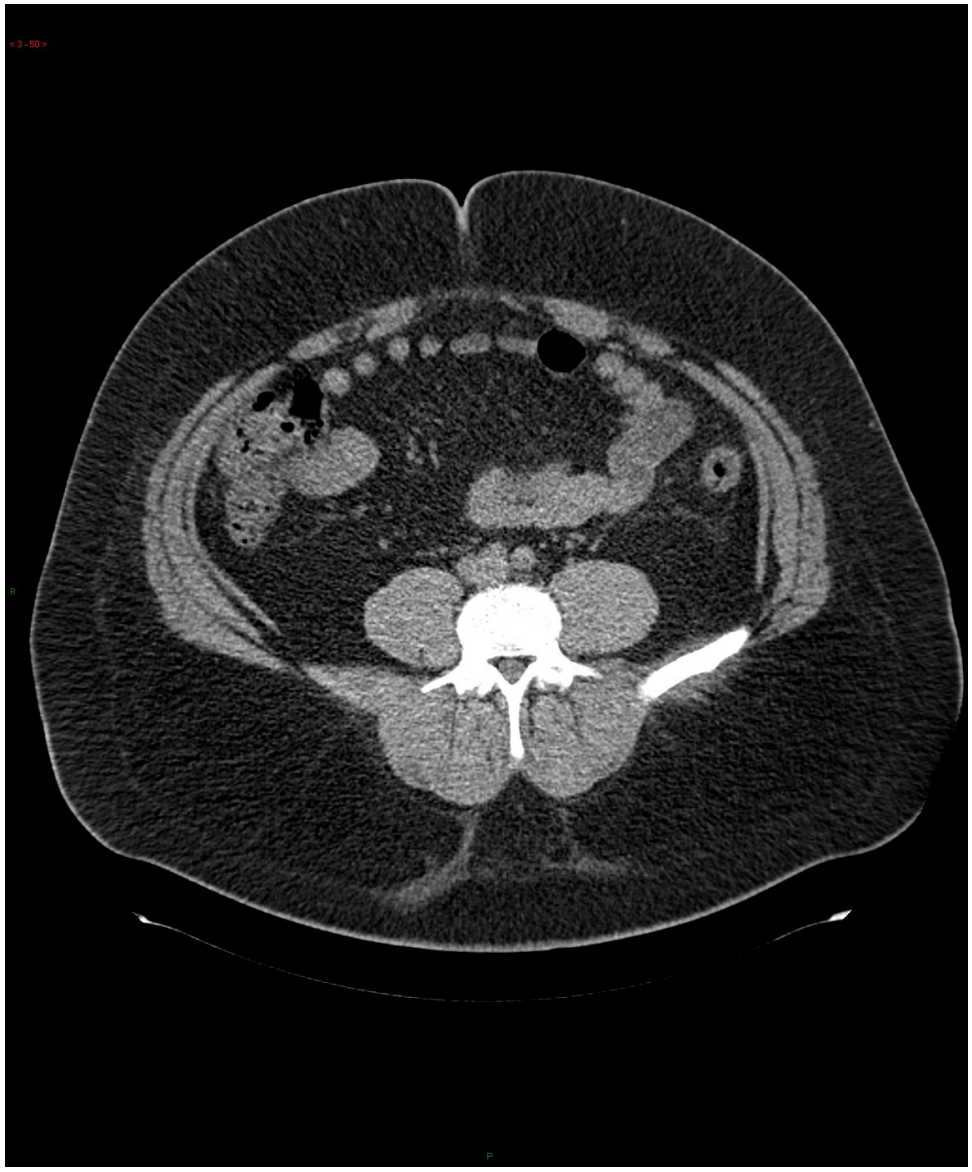
Goal: Detect the heart beat rate.

Note: For 1D sequential data, RNN's are used more often.

Andrew Ng

3D data

3D CT Scan



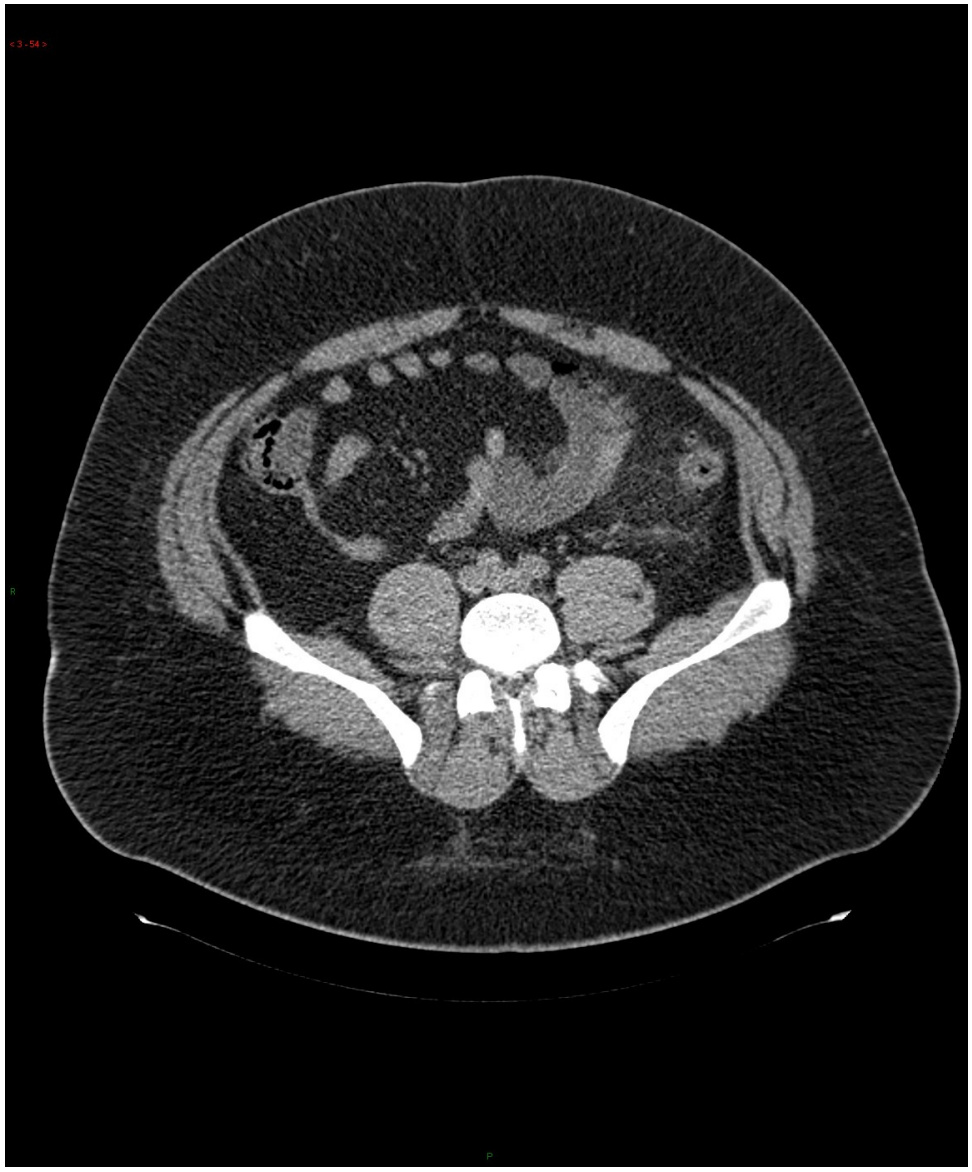
Andrew Ng

3D data



Andrew Ng

3D data



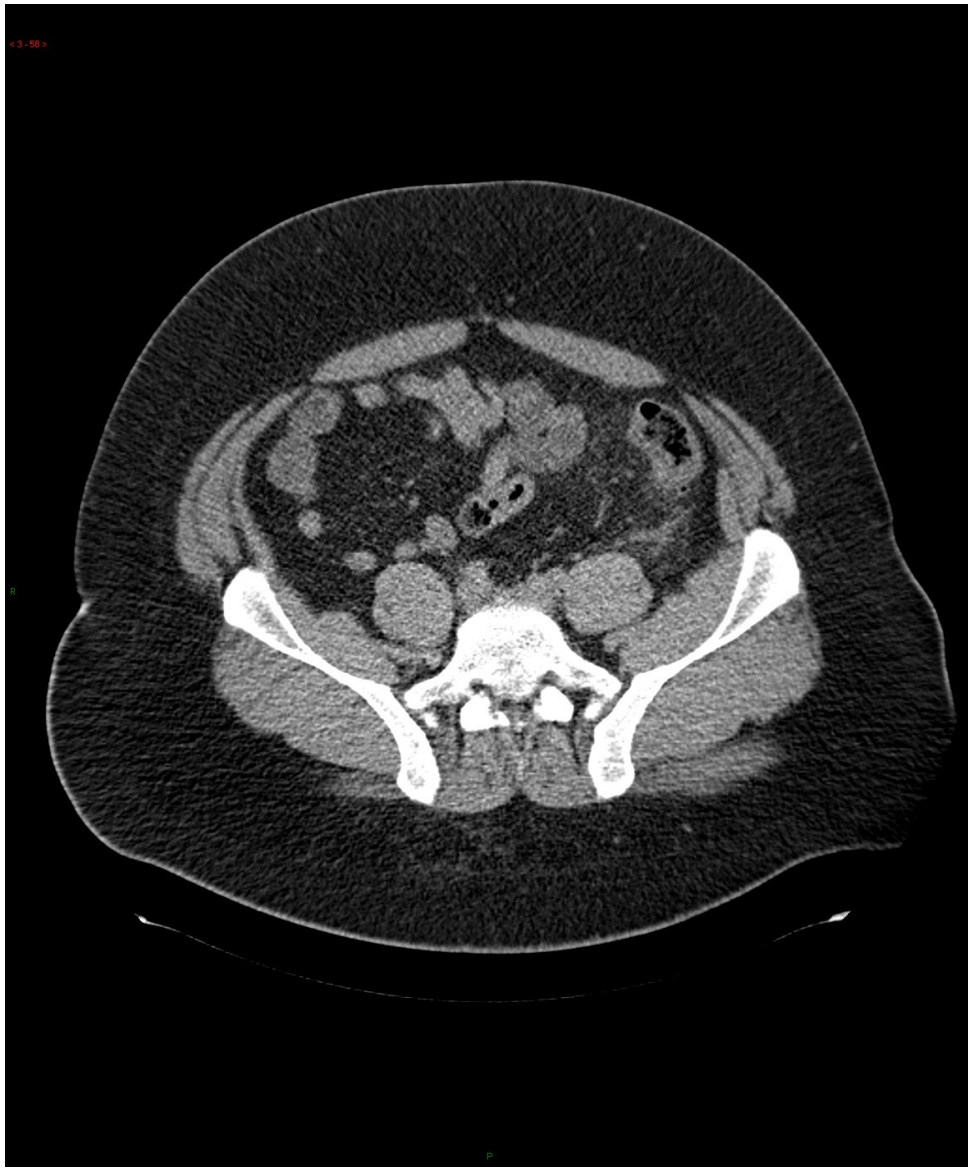
Andrew Ng

3D data



Andrew Ng

3D data



Andrew Ng

3D data



Andrew Ng

3D data



Andrew Ng

3D data



Andrew Ng

3D data



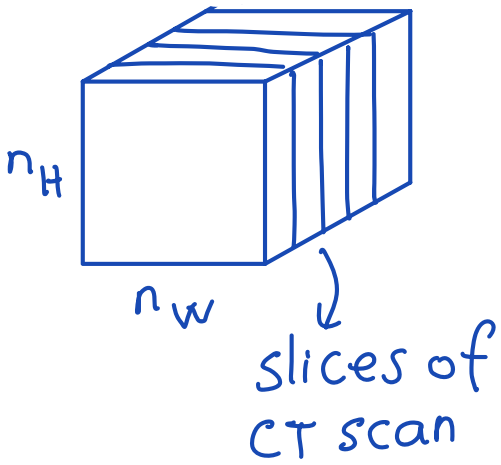
Andrew Ng

3D data



Andrew Ng

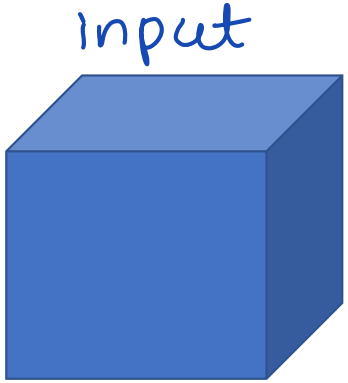
3D data





Andrew Ng

3D convolution





*

3D filter : learn "features" across 3D data.

3D volume

output

$14 \times 14 \times 14 \times 1$ (1 $\downarrow$ n_c) $\ast$ $5 \times 5 \times 5 \times 1 \longrightarrow 10 \times 10 \times 10 \times 16$

16 filters

$10 \times 10 \times 10 \times 16$ $\ast$ $5 \times 5 \times 5 \times 16 \longrightarrow 6 \times 6 \times 6 \times 32$

32 filters

3D data: CT, PET, MRI scans (Medical data)

Movies : detect actions in movies etc.

Andrew Ng